Project Proposal: The Verified Market Simulator

Financial Computing & Algo Trading Section

Spring 2026

1 Executive Summary

We propose the development of a **Verified Limit Order Book (LOB) Simulator** built in **OCaml**. This high-performance simulation engine is designed to support advanced research in Market Microstructure, Signal Construction, and Algorithmic Strategies by PhD and MSc students, while serving as a rigorous platform for undergraduate theses.

Unlike existing simulators built in Python or C++, this project leverages **Functional Programming (FP)** and **Formal Verification** to eliminate "state drift" and guarantee regulatory correctness—specifically ensuring properties such as *Fairness*, *Uniformity*, and *Individual Rationality* as defined in recent literature [2].

2 Strategic Justification: Why Build This?

2.1 Enabling Reproducible Research (PhD & MSc)

Currently, students working on microstructure research often build ad-hoc simulators. These are prone to subtle implementation errors that invalidate results.

- **The Goal:** Provide a standardized, high-performance engine where students can plug in their strategies.
- **Impact:** Shifts student focus from "debugging the matching engine" to "researching the signal." This enables higher-quality publications and thesis work in microstructure and algorithmic trading.

2.2 Pedagogical Value & UG Theses

This project is an ideal vehicle for final-year undergraduate theses, combining financial theory with rigorous software engineering.

- **Interdisciplinary Scope:** It bridges Financial Computing (matching logic), Software Engineering (HPC/Performance), and Theoretical Computer Science (Formal Verification).
- **Current Interest:** We already have five undergraduate applicants interested in this specific intersection of finance and functional programming.

3 Technical Justification: Why Functional Programming?

Anticipating the question: "Why not use C++, Rust, or Python?"

3.1 The Problem: State Drift in Imperative Simulators

Financial simulators built in Python—like those developed independently by students Konark Jain and Antonio Briola—are excellent prototypes but suffer from limitations in rigorous backtesting contexts.

- In imperative languages (Python/C++), market state is mutated in place (`OrderBook.update()`).
- Subtle bugs or race conditions can cause the simulated state to diverge from reality over time, invalidating backtest results.
- Debugging these divergences is notoriously difficult because the "history" of the state is lost after mutation.

3.2 The Solution: Immutability & Exact Replay

By using a **Purely Functional** approach (OCaml), we treat every market state as an **immutable snapshot**.

- **Update Logic:** $update(OldBook, Order) \rightarrow NewBook$. The old state is preserved.
- **Time-Travel Debugging:** We can step backward and forward through every tick of the simulation to identify exactly where a strategy failed.
- **Elimination of Bug Classes:** Immutability eliminates race conditions and state drift by design. This approach allows us to formally define and prove market invariants (e.g., that a matching is always *Fair* on bids and asks) using theorem provers, a methodology pioneered in *Formally Verified Trades in Financial Markets* [2].

3.3 Why OCaml?

While Haskell is often used in academia, **OCaml** is the optimal choice for this specific project due to its proven track record in high-frequency trading:

- **Industrial Validation:** As detailed by Minsky and Weeks [1], OCaml has been successfully deployed at firms like Jane Street for critical trading systems. They found it allowed for rapid development of correct, efficient code, handling billions of dollars in daily transactions.
- **Performance:** OCaml compiles to native code with performance competitive with C++, unlike Python.
- **Multicore Support:** With OCaml 5.0, we can leverage the HPC cluster to run parallel backtests efficiently.
- **Verification Ready:** OCaml's functional nature maps directly to mathematical proofs, enabling Property-Based Testing (QCheck) to verify the complex auction properties required by regulators.

4 Data Compliance & Operational Feasibility

We are aware of the strict licensing constraints regarding LOBSTER data redistribution.

- **Constraint:** We cannot create a central, persistent repository of LOBSTER data for all students.
- **Pipeline Design:** The system is designed as a "Bring Your Own Data" pipeline.

- **Workflow:**
  1. A user (student/researcher) downloads their own LOBSTER data sample under their license.
  2. They run our local **Preprocessor Tool** (built in Phase 1) to convert the CSVs into a temporary, high-efficiency binary format for the simulator.
  3. The simulation runs on the HPC.
  4. The raw data can be discarded or stored locally by the user, ensuring no central licensing breach.

5 Conclusion

This project is not just a coding exercise; it is an infrastructure investment. By building a verified, high-performance simulator now, we create a lasting foundation for the next generation of Algo Trading and Microstructure classes, ensuring our students work with tools that meet the rigorous standards of modern quantitative finance.

References

[1] Yaron Minsky and Stephen Weeks. *Caml trading: experiences with functional programming on Wall Street*. Journal of Functional Programming, 18(4):553–564, 2008.

[2] Suneel Sarswat and Abhishek Kr Singh. *Formally Verified Trades in Financial Markets*. In *Formal Methods and Software Engineering (ICFEM 2020)*, pages 217–232. Springer, 2020.